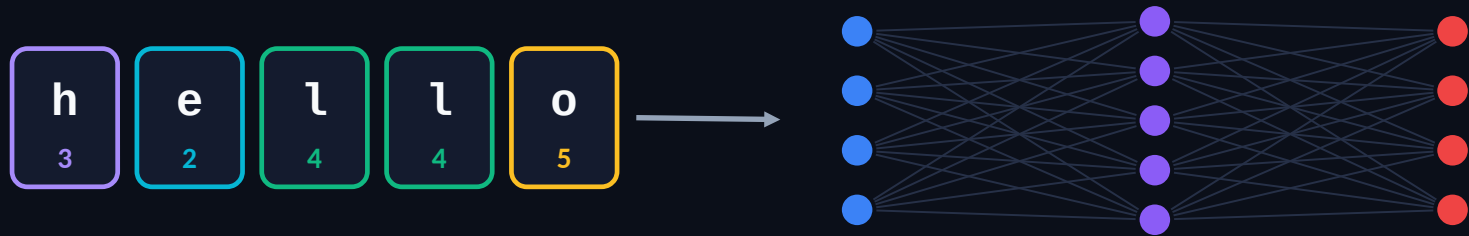


How Next-Token Batching Works

Preparing training examples for a transformer



text → tokens → neural network

Start With Text

STEP 1 / 10

"hello world"

Computers can't read letters. Before a language model can do anything with this sentence, every character has to become a number.



11 characters → 11 unknown numbers (so far)

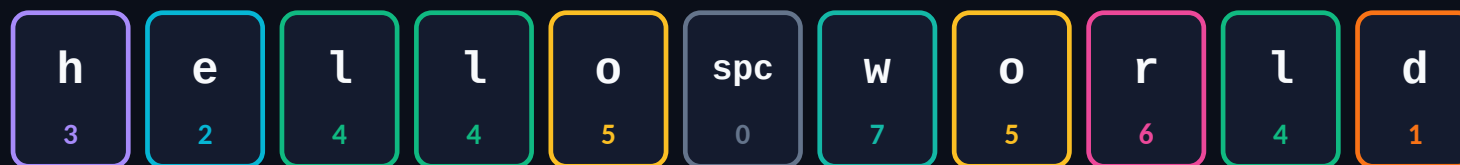
Tokenize the Text

STEP 2 / 10

Each character gets its own fixed number, called a token ID. This mapping is the vocabulary.



"hello world" →



[3, 2, 4, 4, 5, 0, 7, 5, 6, 4, 1]

This numeric sequence is what the model will actually train on.

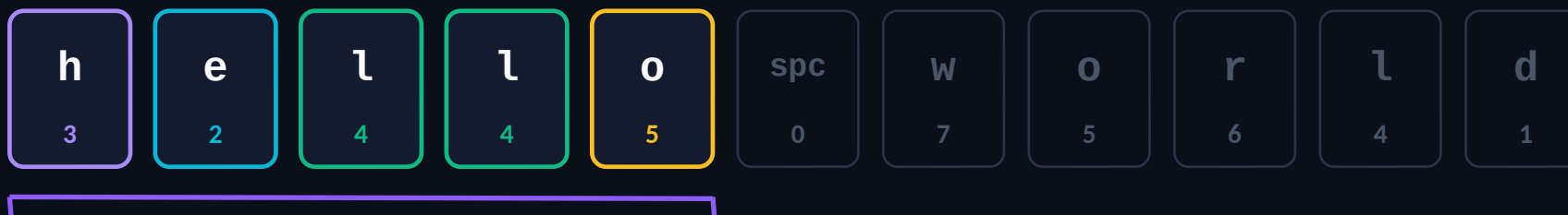
Choose a Block Size

STEP 3 / 10

4

block_size

block_size is the maximum number of tokens in one training example. To build one input and its targets, we actually need block_size + 1 tokens from the data.



5 tokens = block_size (4) + 1 extra

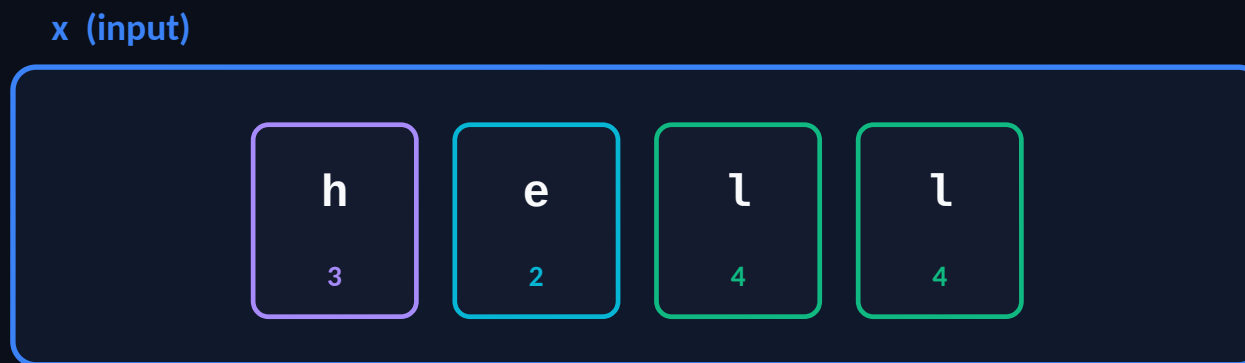
The bright tokens are the working slice for this example. The dimmed tokens aren't needed yet.

Create the Input

STEP 4 / 10

```
x = data[0:4]
```

Take the first `block_size` tokens from the sequence. This slice is what the model will see.

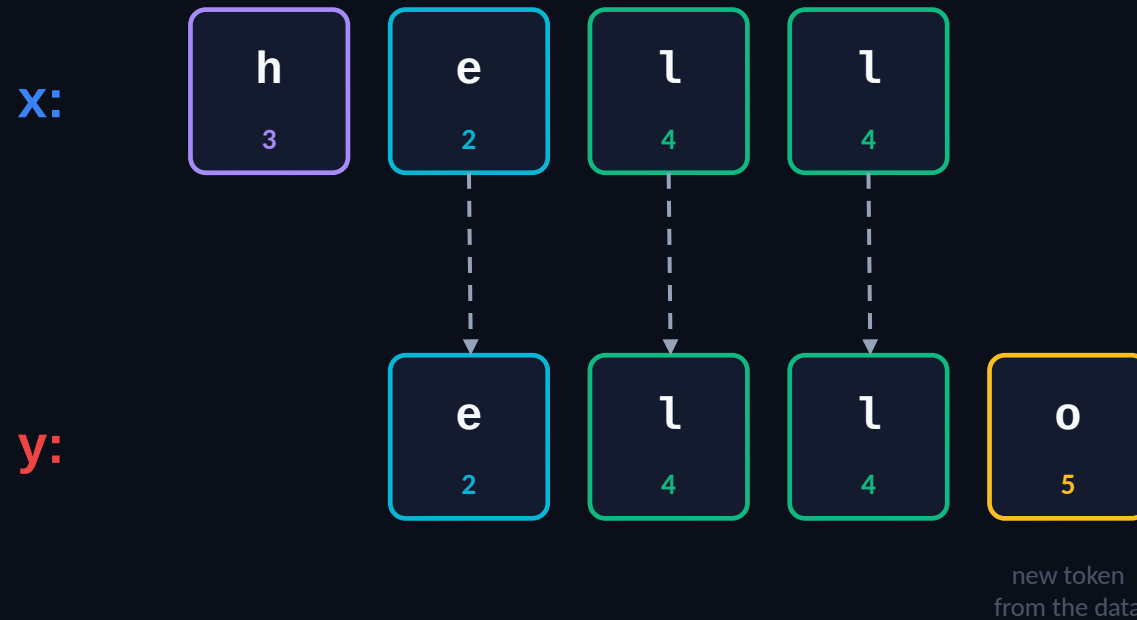


Input sequence — what the model is given to make a prediction.

Create the Shifted Target

STEP 5 / 10

`y = data[1:5]`

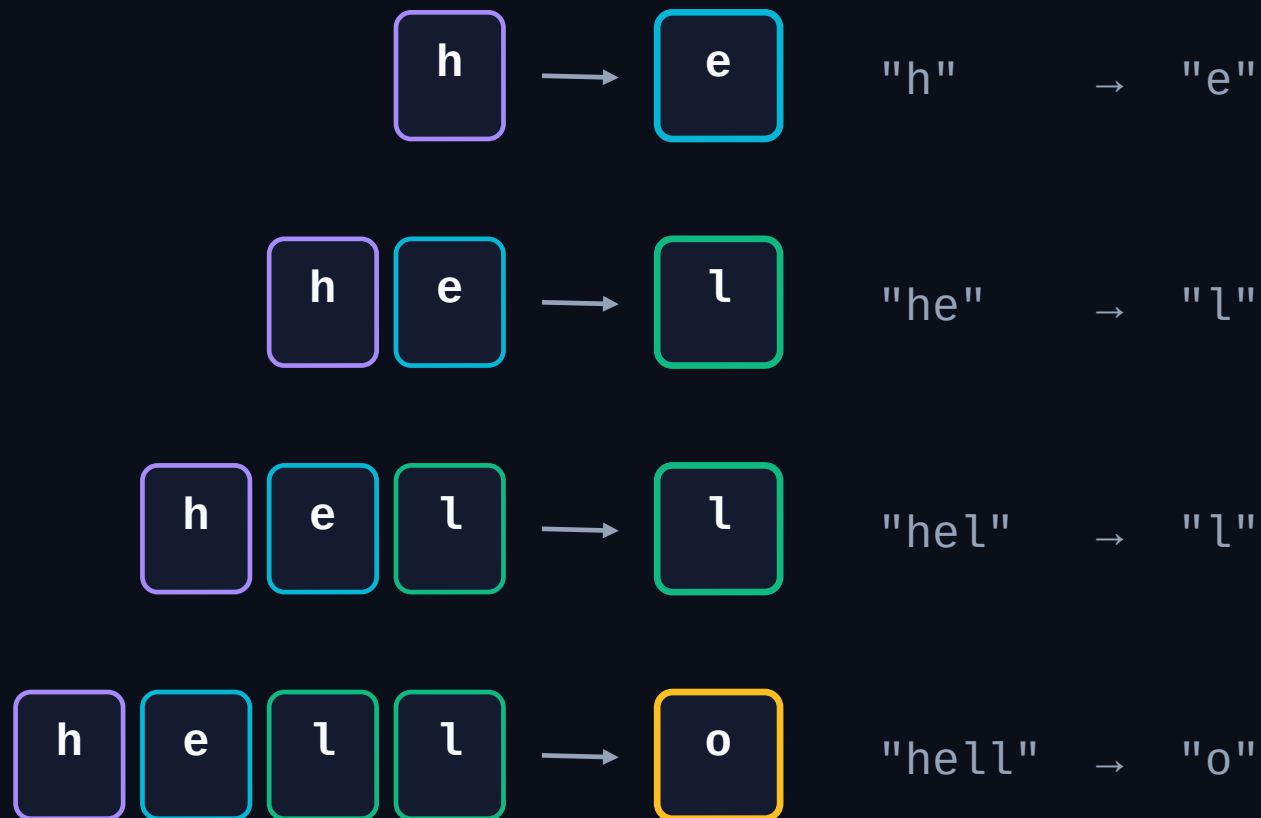


y is x shifted one token forward. Whatever comes right after a token in the data is that token's target.

What the Model Learns

STEP 6 / 10

Every input/target pair actually hides several smaller lessons — one per position.



The model learns to predict the next token using everything that came before it.

Add a Batch

STEP 7 / 10

2
batch_size

4
block_size

x

h 3	e 2	l 4	l 4
spc 0	w 7	o 5	r 6

shape (2, 4)

y

e 2	l 4	l 4	o 5
w 7	o 5	r 6	l 4

shape (2, 4)

- Each row is an independent sequence, pulled from a different random spot.
- batch_size is the number of rows.
- block_size is the number of tokens in each row.
- x and y are both tensors of shape (batch_size, block_size).

Walk Through the Code

STEP 8 / 10

```
import torch
```

```
class NextTokenBatcher:
```

```
    """Create input and target batches."""
```

```
    def __init__(self, train_data, val_data,
                  block_size, batch_size):
```

```
        self.train_data = train_data
```

```
        self.val_data = val_data
```

```
        self.block_size = block_size
```

```
        self.batch_size = batch_size
```

```
    def get_batch(self, split):
```

```
        # returns a batch of (x, y)
```

```
        data = self.train_data if split == "train"
        else self.val_data
```

```
        ix = torch.randint(
            len(data) - self.block_size,
            (self.batch_size,))
```

```
        x = torch.stack([data[i:i + self.block_size]
                          for i in ix])
```

```
        y = torch.stack([data[i+1:i+1 + self.block_size]
                          for i in ix])
```

```
        return x, y
```

1

Choose the data

Pick the train or validation split

2

Pick starting points

Random offsets into that data

3

Slice the inputs — x

block_size tokens, stacked into a batch

4

Slice the targets — y

Same slice, shifted one token later

5

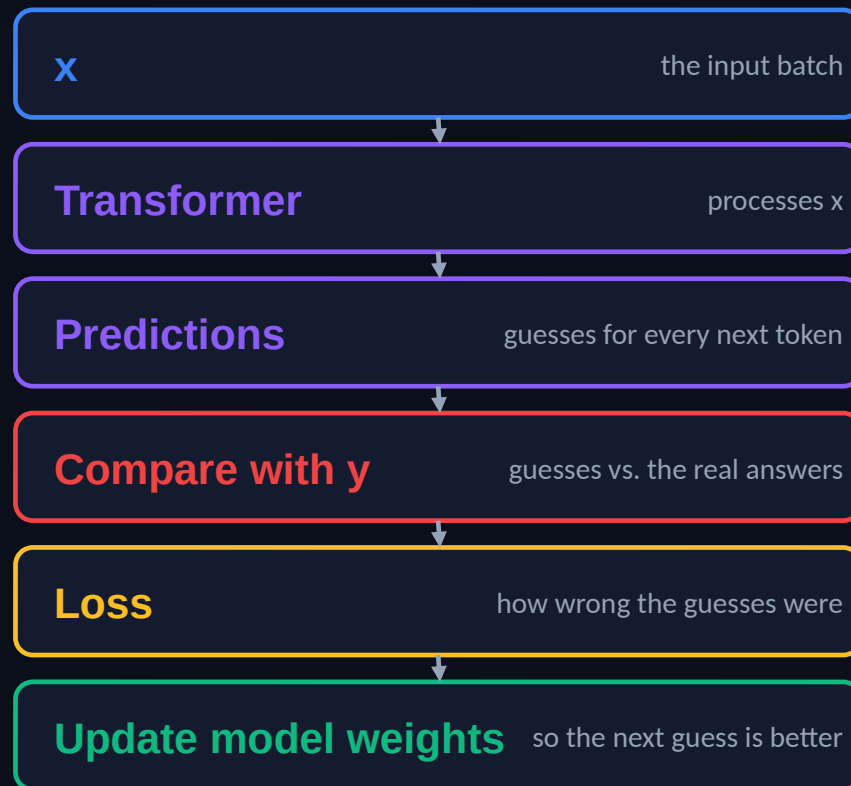
Return the batch

Hand x and y to the training loop

What Happens Next?

STEP 9 / 10

The batcher's job ends here. Training itself is a separate loop.



x goes into the transformer. y is never shown to the model — it's only used afterward, to check how close the predictions were.

Knowledge Check

STEP 10 / 10

If the next token after this sequence is 50, what should y be?

$x = [10, 20, 30, 40]$



ANSWER

$y = [20, 30, 40, 50]$



Summary

1

Randomly select a section of
tokenized text.

2

Use the selected tokens as x.

3

Shift forward one token to create
y.

“The batcher prepares **the question** and **the answer**. The transformer
learns how to predict
the answer.”